

CentiSible — Diário de Bordo & Decisões

O processo mental, a arquitetura de escolhas e os bastidores do desenvolvimento

Autor: António (Desenvolvedor) | Estado do Projeto: Em Produção (Render + Neon + Vercel) | Última Atualização: 02 de Setembro de 2026

Olá! Se pegaste neste projeto ou estás a avaliar a minha defesa, este documento é para ti.

A ideia aqui não é listar código ou repetir a documentação técnica (isso já está no repositório e no `ARCHITECTURE.md`). O objetivo deste registo é explicar o **meu Raciocínio Mental**: porque tomei certas decisões, as alternativas que pondera e recusei, os bugs reais que me apareceram pelo caminho em produção e como os resolvi. É a história de como o projeto evoluiu de uma ideia simples até a uma aplicação robusta, rápida e pronta a usar.

1. ESTADO ATUAL & RESUMO OPERACIONAL

COMPONENTE	INFRAESTRUTURA / TECNO	ESTADO PRÁTICO
Backend (API)	FastAPI (Python 3.12) na Render	252 Testes Pytest OK centisible.onrender.com
Base de Dados	PostgreSQL gerido na Neon	12 tabelas com migrações Alembic ativas.
Frontend & PWA	React + Vite + Tailwind na Vercel	PWA Instalável centisible.vercel.app
Testes & Qualidade	Pytest + Playwright E2E + CI	CI GitHub Actions Verde 9 specs E2E a passar.
Identidade Visual	Paleta "Oliveira" (Verde + Âmbar)	Design responsivo com tema escuro/claro e animações.

2. CRONOLOGIA DE DESENVOLVIMENTO & RACIOCÍNIO POR DETRÁS DAS DECISÕES

02 DE SETEMBRO DE 2026

Consolidação Técnico-Comercial (PITCH.md e Documento Técnico)

Senti necessidade de fechar a comunicação do projeto para a apresentação ao júri. Reescrevi o discurso de abertura (`PITCH.md`) para ser direto, falado na primeira pessoa e sem jargão excessivo: explica o problema (o equilíbrio perfeito entre folhas de cálculo maçadoras e apps de finanças demasiado complexas), a solução (gestão inteligente com alertas e agregado com privacidade) e a prova de execução real.

Paralelamente, consolidei todo o conhecimento disperso no `DOCUMENTO-TECNICO.md`. O foco foi ter um entregável único que sintetiza a arquitetura, modelo de dados, decisões técnicas e a resolução dos maiores desafios encontrados.

01 DE SETEMBRO DE 2026

Despesas Partilhadas do Agregado: Modelo "Lançada Uma Vez" e Prevenção de Duplicados

O Problema: Ao testar o agregado com duas contas reais, lancei a renda da casa de 450€ em ambos os perfis. O painel somava 900€! O modelo antigo assumia que "cada um lança a sua fração", mas na prática as pessoas esperam que uma despesa da casa lançada por um membro seja a única.

A Decisão: Mudei para o modelo *"uma despesa partilhada é um custo da casa, lançado uma só vez por quem pagou"*. Não quis fundir silenciosamente os dados porque isso destruiria casos legítimos onde o casal divide formalmente o custo em frações diferentes.

A Solução Técnica: Se outro membro já registou uma despesa partilhada igual (mesmo valor + mesma categoria + mesmo mês), o backend recusa com `HTTP 409 Conflict` nomeando quem já a lançou. O frontend mostra um modal com duas opções claras: *"Lançar à mesma"* (reenvia com `?allow_duplicate=true`) ou *"Rever"*.

01 DE SETEMBRO DE 2026

Categorias-Padrão Automáticas ao Registrar Conta

Reparei num atrito grave no primeiro minuto de utilização: um utilizador acabadinho de criar conta apanhava o painel e os formulários de transação completamente vazios, sem saber por onde começar. A solução foi injetar automaticamente ~11 categorias iniciais (Alimentação, Habitação, Saúde, Salário, etc.) com cores e ícones atribuídos logo no momento do registo (dentro da mesma transação do banco de dados), garantindo que a app fica pronta a usar imediatamente.

31 DE AGOSTO DE 2026

Resiliência em Produção: Tolerância a Concorrência de Tokens e Recuperação de Password

Bug de Sessão Morta: Em produção, quando o servidor adormecia (plano gratuito da Render) ou quando o utilizador tinha a PWA e uma aba do browser abertas em simultâneo, os refreshes concorrentes acionavam o sistema de deteção de roubo de tokens, deslogando a pessoa injustamente.

Como Resolvi:

- **Janela de Graça (Grace Window):** No backend, introduzi uma tolerância de 10 segundos na rotação de refresh tokens. Se o mesmo token for reutilizado num intervalo inferior a 10s, o servidor devolve um aviso `409` em vez de revogar a conta inteira por falso alarme de roubo.
- **Lock no Browser:** Implementei `navigator.locks` no cliente HTTP para garantir que só um pedido de renovação sai de cada vez no mesmo dispositivo.
- **Recuperação de Password Segura:** Criei um fluxo completo com tokens temporários (hash SHA-256) e integração com a API do Resend para envio de emails reais.
- **Cabeçalhos de Segurança HTTP:** Adicionei CSP estrita, HSTS, X-Frame-Options e SameSite reforçado contra potenciais ataques.

30 DE AGOSTO DE 2026

Transformação em PWA (Progressive Web App) & Ajustes Mobile

Para tornar a experiência o mais próxima possível de uma aplicação nativa sem a complexidade desnecessária de publicar nas app stores, converti o frontend numa PWA utilizando o `vite-plugin-pwa`. Configurei o manifesto, gerei os ícones maskable para Android/iOS e estruturei o Service Worker com estratégia `NetworkOnly` para chamadas de API (evitando assim mostrar dados financeiros obsoletos em cache).

Aproveitei para otimizar o layout mobile do painel: reordenei os elementos para que o conteúdo urgente (Insights e Próximos Pagamentos) apareça logo após os cartões de saldo, enquanto gráficos mais analíticos passaram para separadores inferiores.

29 DE AGOSTO DE 2026

Evolução de Marca: De FinTrack para "CentiSible" e Nova Paleta "Oliva"

Rebranding: Mudei o nome do projeto de "FinTrack" para **CentiSible** (junção de "*cent*" com "*sensible*"). Criei um logótipo próprio: um "C" estruturado com um gráfico de crescimento e uma moeda de cêntimo no interior.

Paleta de Cores "Oliva": Abandonei a combinação comum de violeta/teal e adotei um tom verde-floresta profundo (representando estabilidade financeira) com acentos raros em âmbar quente. Esta escolha resolveu um conflito visual grave: antes, as cores da marca confundiam-se com o verde de "receita" e vermelho de "despesa".

Melhorias de Interface: Redesenhei a listagem de transações com agrupamento diário (Hoje, Ontem, etc.), adicionei um modal centrado de detalhe (evitando problemas de scroll em telemóveis) e implementei o carregamento de recibos/comprovativos com armazenamento em volume isolado.

27 DE AGOSTO DE 2026

Implementação em Massa da Lógica de Negócio (Fases 6 à 13)

Nesta sessão integrei a maioria dos motores financeiros do sistema:

- **Orçamentos (Budgets) & Alertas:** Cálculo de consumo limite por categoria com avisos visuais aos 80% e 100%.
- **Despesas Recorrentes:** Motor para gerir subscrições/contas fixas (ex: Netflix, Renda) com geração manual ou agendada de transações.
- **Objetivos Financeiros (Goals):** Sistema de metas de poupança com barra de progresso calculada em tempo real.
- **Histórico & Analytics:** Gráficos comparativos de receitas vs. despesas e evolução mensal via Recharts.
- **Agregado Familiar (Households):** Sistema de convites com código único, permitindo a fusão opcional de dados sem comprometer a privacidade das contas pessoais.

3. O QUE APRENDI & PRINCÍPIOS QUE DEFENDO

- 1. Tratar Erros na Fonte:** Substituí 58 blocos repetitivos de `try/except` nos routers do FastAPI por um único `DomainError exception_handler` global. Isso removeu mais de 250 linhas de código duplicado e centralizou todas as mensagens do sistema num só ficheiro.
- 2. Menos É Mais no Estado da UI:** O frontend não tenta adivinhar dados antigos. Uso React Query com `keepPreviousData` para transições suaves entre meses sem que os ecrãs pisquem em branco.
- 3. Testes como Rede de Segurança Real:** A existência de mais de 250 testes automatizados no backend e testes E2E com Playwright permitiu-me reescrever partes inteiras da UI e refazer a rotação de tokens com a confiança de que nada no ecossistema se quebrava.

CentiSible — Desenvolvido por António | Documento gerado automaticamente para efeitos de apresentação e defesa